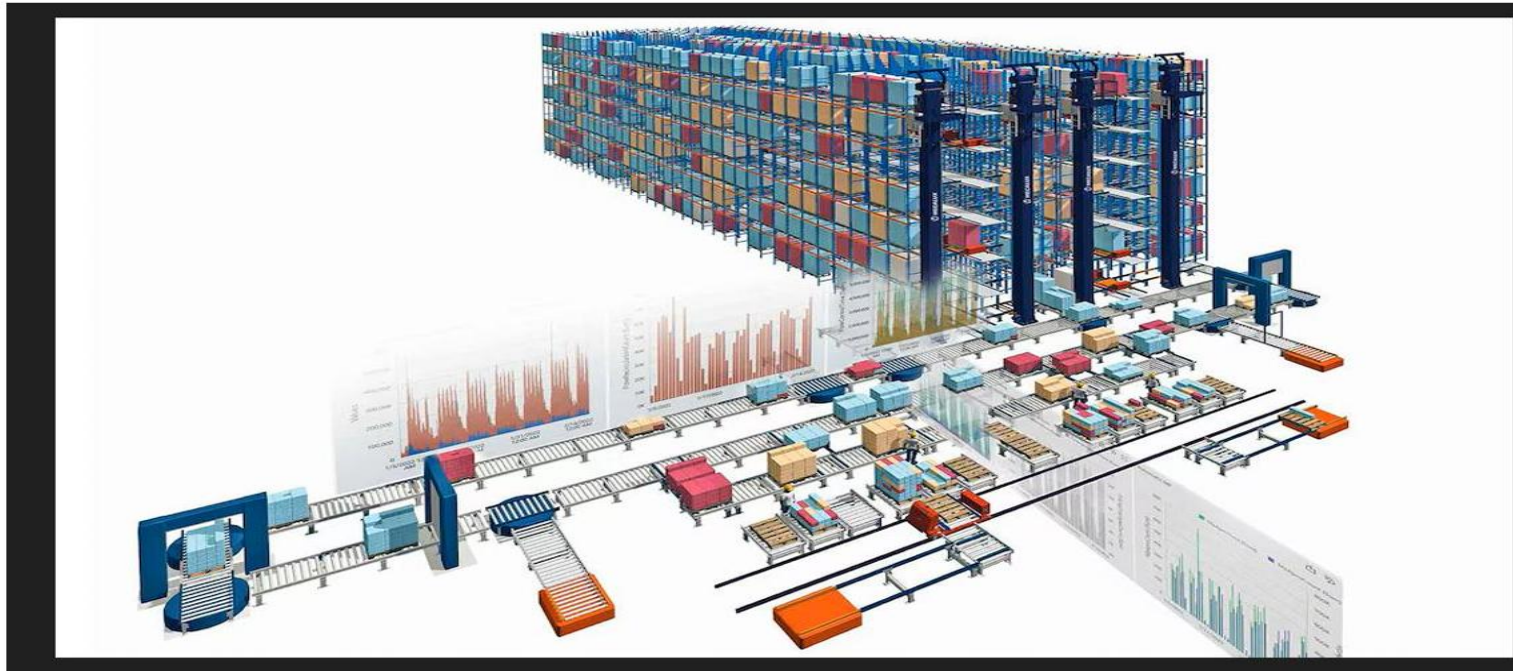


Module : Algorithmes avances



Dr Sada ANNE
sada.anne@uadb.edu.sn

Année académique: 2024-2025

Chapitre 4: Algorithmes de tri

1. Introduction
2. Tri par sélection
3. Tri par insertion
4. Tri à bulle
5. Tri rapide
6. Tri par fusion

Introduction

- Le terme "trier" signifie «répartir des objets suivant certains critères».
- En algorithmique, le terme "tri" est souvent attaché au processus de classement d'une suite d'éléments dans un ordre donné.

Exemple:

- Trier N entier dans un ordres croissant.
- Trier N étudiants dans l'ordre décroissant de leurs moyennes (réel).
- Trier N noms dans l'ordre alphabétique croissant.
- Trier des fichiers selon leurs tailles.

Introduction

- D'une façon générale, tout ensemble muni d'un ordre total peut fournir une suite d'éléments à trier.
- Deux catégories de tris :
 - **Les tris internes** : méthodes destinées à des masses de données limitées, stockées dans une structure de données se trouvant dans la mémoire centrale (Exemple : tableaux).
 - **Les tris externes** : méthodes destinées à de grandes masses de données, stockées dans des structures de données telle que les fichiers.

Introduction

- Le tri est fondamental à beaucoup d'autres problèmes, et après le tri, beaucoup de problèmes deviennent faciles à résoudre. par exemple :
 - La recherche d'un éléments.
 - Unicité d'éléments: après le tri tester les éléments adjacents.
 - Déterminer le plus petit et le plus grand élément.

Tri par sélection

Principe:

Répéter:

- Chercher le plus petit (plus grand) élément => **Sélection**.
- Mettre l'élément sélectionné au début de la partie non triée.

D'une autre manière

- Trouver le plus petit élément et le mettre au début du tableau
- Trouver le 2e plus petit éléments et le mettre en seconde position
- Trouver le 3e plus petit éléments et le mettre à la 3e place,
- ...

Tri par sélection

Exemple: T

10	12	25	30	2	17	4
----	----	----	----	---	----	---

min = 2

Etape 1: Faire l'échange entre 2 (min) et 10

2	12	25	30	10	17	4
---	----	----	----	----	----	---

min = 4

Etape 2: Faire l'échange entre 4 (min) et 12

2	4	25	30	10	17	12
---	---	----	----	----	----	----

min = 10

Tri par sélection

Etape 3: Faire l'échange entre 10 (min) et 25



Etape 4: Faire l'échange entre 12 (min) et 30



Etape 5: Faire l'échange entre 17 (min) et 25



Etape 6: 25 (min) est à sa palce



Tri par sélection (Algorithme)

Procédure TriSelection (Var T : Tableau d'entiers, N : entier)

i, j, indMin, temp : entier;

Début

Pour i allant de 1 à N-1 faire

indMin = i;

Pour j allant de i+1 à N

si ($T[j] < T[indMin]$) alors

indMin = j;

Fin Si

Fin Pour

temp = t[i];

T[i] = T[indMin];

T[indMin] = tmp;

Fin pour

Fin

**/* recherche de l'indice
du minimum */**

**/* échange des valeurs entre
la case courante et le
minimum */**

Tri par sélection (Algorithme)

Procédure TriSelection (Var T : Tableau d'entiers, N : entier)

 i, j, indMin , temp : entier;

Début

 Pour i allant de 1 à N-1 faire

 indMin $\leftarrow$ indiceMin (T, i, N)

 échanger (T, i, indMin);

 Fin pour

Fin

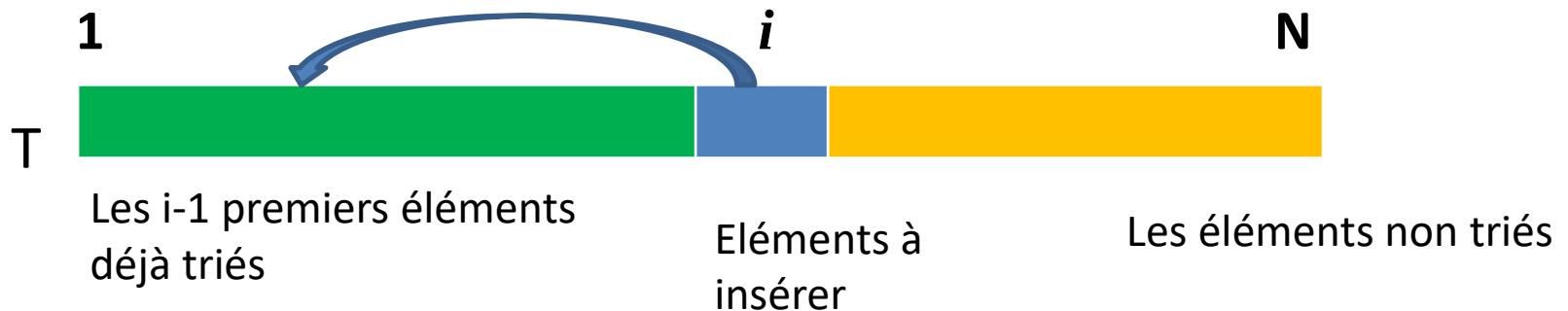
- La fonction indiceMin retourne l'indice du minimum de la partie du tableau [i, N]
- La fonction échanger fait la permutation entre l'élément i et le minimum

Tri par Insertion

Principe:

Répéter:

- Insertion du prochain élément dans la partie qui est déjà triée précédemment.
- La partie de départ qui est triée est le premier élément.
- Il se pourrait qu'on a à déplacer plusieurs éléments pour l'insertion.



Tri par Insertion

Principe:

(le tri du joueur de cartes !)

- Ordonner les deux premiers éléments
- Insérer le 3e élément de manière à ce que les 3 premiers éléments soient triés
- Insérer le 4e élément à “sa” place pour que. . .
- . . .
- Insérer le *nième* élément à sa place.

A la fin de la *i* ème itération, les *i* premiers éléments de T sont tries.

TAB = {4,1,3, 2};

Tri par Insertion

Exemple:

T

10	12	5	30	2	17	4
----	----	---	----	---	----	---

Etape 1: insérer 12 à sa place

10	12	5	30	2	17	4
----	----	---	----	---	----	---

Etape 2: insérer 5 à sa place

5	10	12	30	2	17	4
---	----	----	----	---	----	---

Tri par Insertion

Etape 3: insérer 30 à sa place

5	10	12	30	2	17	4
---	----	----	----	---	----	---

Etape 4: insérer 2 à sa place

2	5	10	12	30	17	4
---	---	----	----	----	----	---

Etape 5: insérer 17 à sa place

2	5	10	12	17	30	4
---	---	----	----	----	----	---

Etape 6: insérer 4 à sa place

2	4	5	10	12	17	30
---	---	---	----	----	----	----

Tri par Insertion (Algorithme)

Procédure TriInsertion (Var T : Tableau d'entiers, N : entier)

 i, k : naturels;

 temp : entier;

Début

Pour i=2 à N faire

 temp $\leftarrow$ t[i]; k $\leftarrow$ i;

 Tant que (k > 1 et t[k-1] > temp) faire

 T [k] $\leftarrow$ T[k - 1];

 k $\leftarrow$ k - 1;

 Fin tant que

 T[k] $\leftarrow$ temp;

Fin pour

Fin

Tri à bulle

Principe:

Répéter:

- Parcourir le tableau en comparant deux à deux les éléments successifs, permuter s'ils ne sont pas dans l'ordre.
 - Répéter tant que des permutations sont effectuées.
-
- Après le premier parcours, le plus grand élément étant à sa position définitive, il n'a plus à être traité.
 - Le reste du tableau est en revanche encore en désordre. Il faut donc le parcourir à nouveau, en s'arrêtant à l'avant-dernier élément.
 - Après ce deuxième parcours, les deux plus grands éléments sont à leur position définitive.
 - Il faut donc répéter les parcours du tableau, jusqu'à ce que les deux plus petits éléments soient placés à leur position définitive.

Tri à Bulle

Exemple: T

12	10	5	30	2	17
----	----	---	----	---	----

Itération 1:

12	10	5	30	2	17
----	----	---	----	---	----

10	12	5	30	2	17
----	----	---	----	---	----

10	5	12	30	2	17
----	---	----	----	---	----

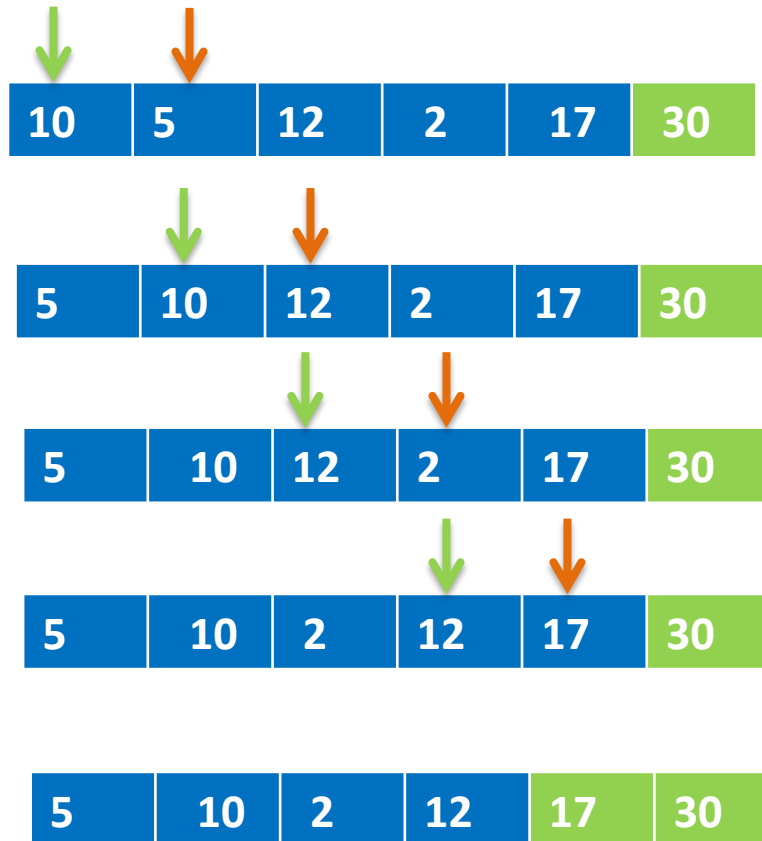
10	5	12	30	2	17
----	---	----	----	---	----

10	5	12	2	30	17
----	---	----	---	----	----

10	5	12	2	17	30
----	---	----	---	----	----

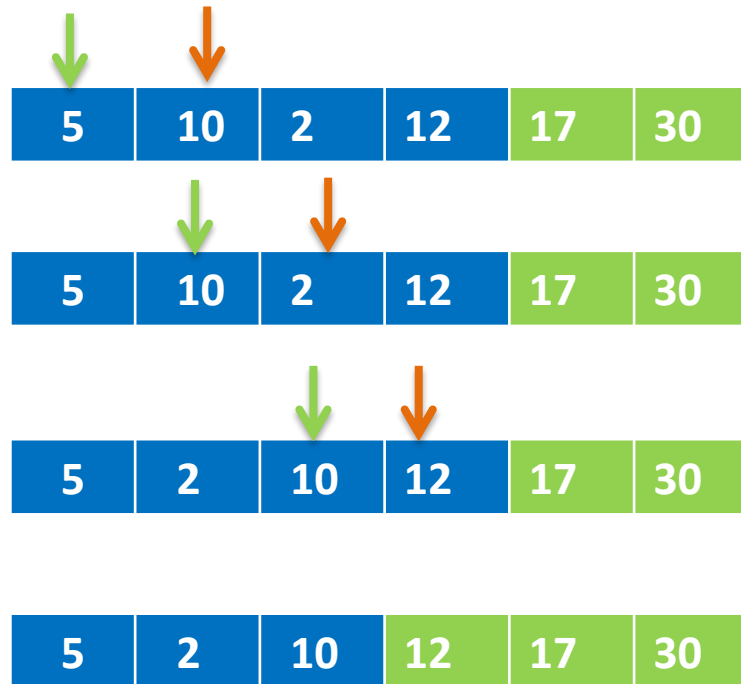
Tri à Bulle

Itération 2:



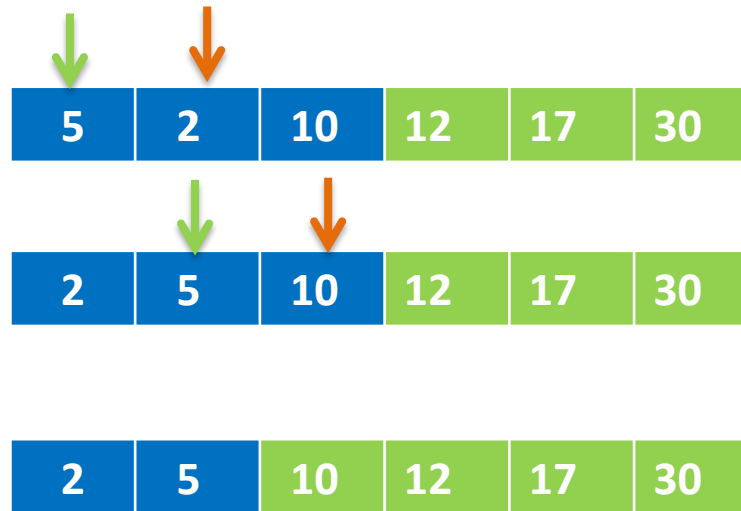
Tri à Bulle

Itération 3:



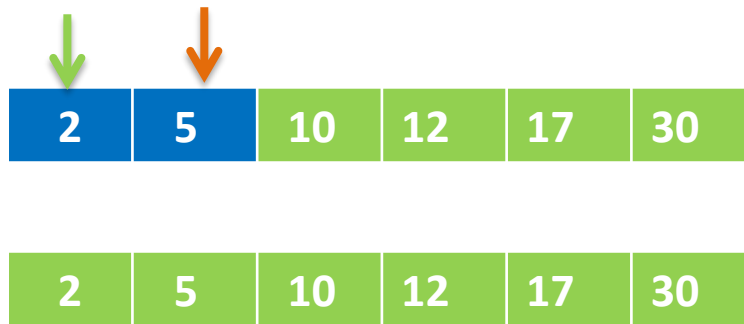
Tri à Bulle

Itération 4:



Tri à Bulle

Itération 5:



Aucun changement arrêt de l'algorithme

Tri à Bulle (Algorithme)

Procédure TriBulle (Var T : Tableau d'entiers, N : entier)

i : naturels;

B: Bouléen;

Début

Répéter

B $\leftarrow$ Faux;

Pour i allant de 1 à N -1 faire

Si T[i] >T[i+1] alors

Echanger (T, i, i+1);

B $\leftarrow$ Vrai;

Finsi

Fin pour

N $\leftarrow$ N-1;

Jusqu'à B=faux

Fin

Tri rapide

Principe:

Inventé en 1960 par Sir Charles Antony Richard Hoare basé sur le paradigme *diviser pour régner* consiste à:

- Choisir un élément « pivot »
- Diviser l'ensemble ou le tableau à deux sous-ensembles
- Un sous ensemble contient les éléments inférieurs au pivot et la deuxième contient les éléments supérieur au pivot.

(Partitionnement)

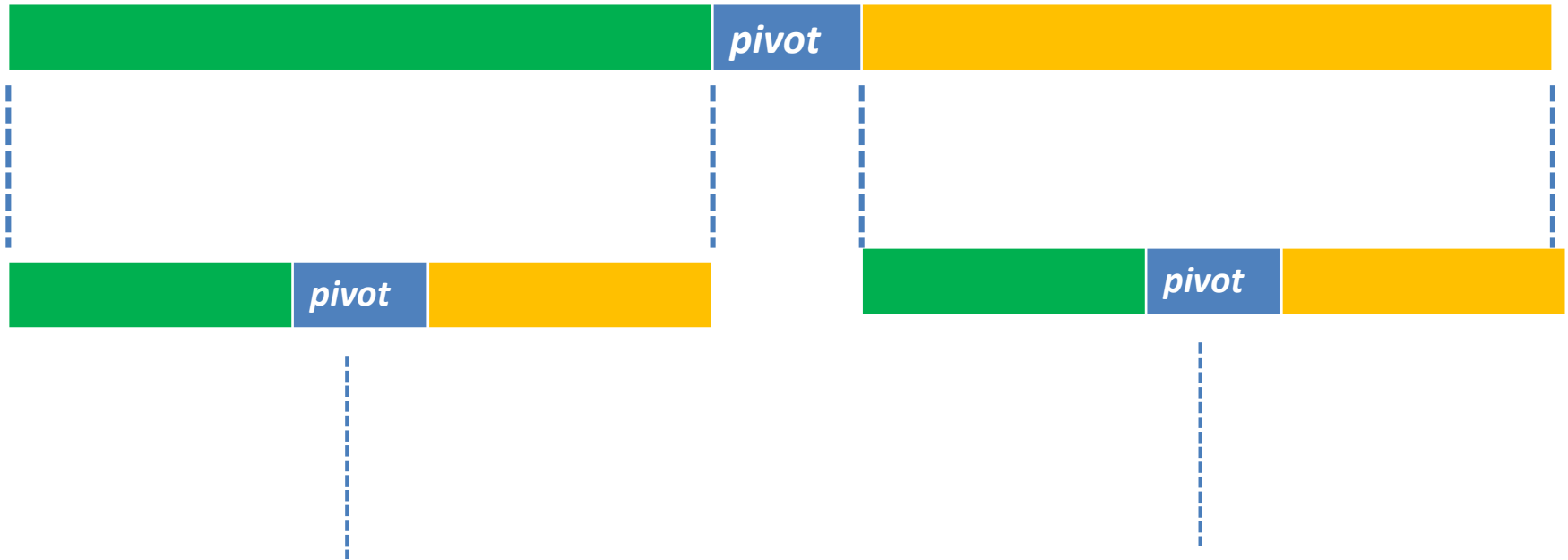
- Répéter la procédure récursivement jusqu'au chaque ensemble contient un seul élément.

Tri rapide

Principe:

Éléments inférieurs au pivot

Éléments supérieurs au pivot

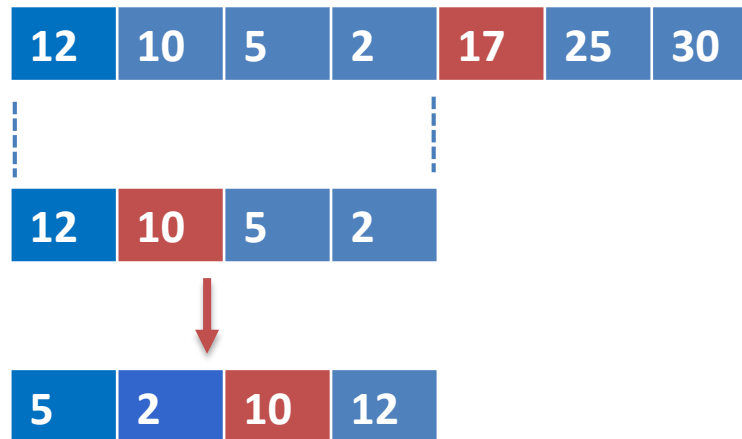


Tri Rapide

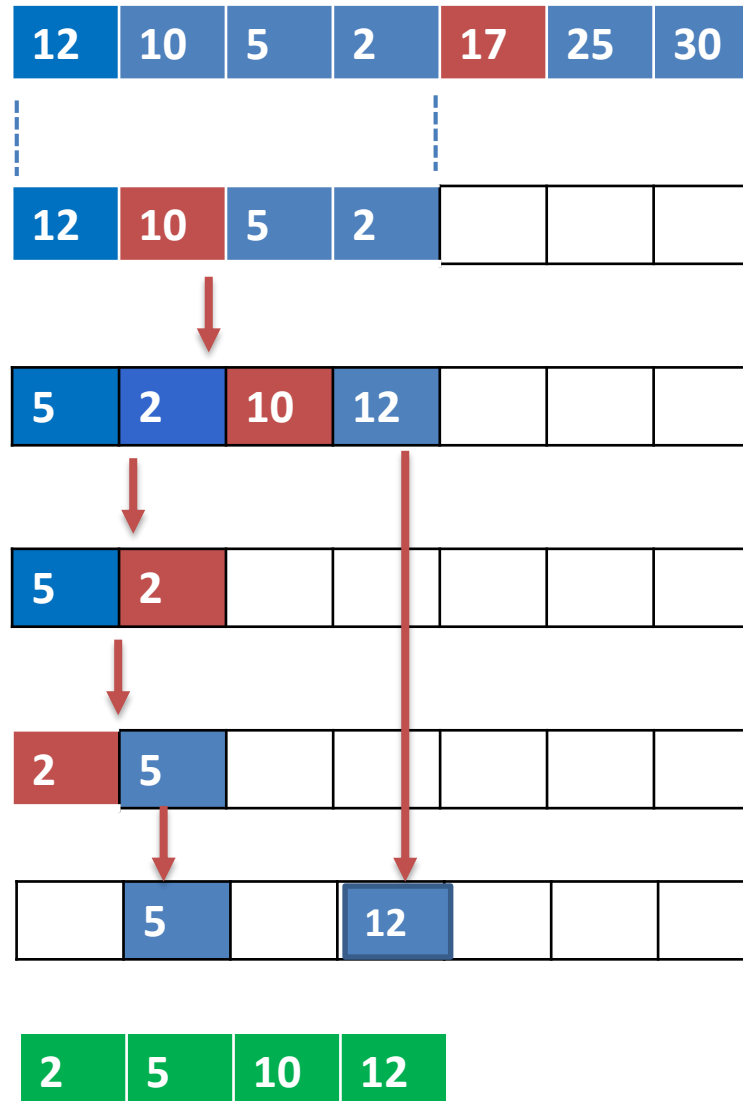
Exemple: T

12	10	5	30	2	25	17
----	----	---	----	---	----	----

- Choisir un pivot (**17**)
- Mettre les éléments <17 dans la partie gauche du pivot et les éléments ≥ 17 dans la partie droite



Tri Rapide



Tri Rapide(Algorithme)

Procédure TriRapide (Var T : Tableau d'entiers, deb, fin: entier)

 pivot: entier

Début

 Si deb < fin alors

 pivot $\leftarrow$ partitionner (T, deb, fin);

 TriRapide (T, deb, pivot -1);

 TriRapide (T, pivot +1, fin);

 Finsi

Fin

Tri Rapide(Algorithme)

Fonction partitionner (Var T : Tableau d'entiers, deb, fin: entier):
entier

pivot: entier

Début

pivot $\leftarrow$ T[(deb + fin) / 2];

i $\leftarrow$ deb; j $\leftarrow$ fin;

Tantque i < j faire

 Tantque T[i] < pivot faire i $\leftarrow$ i+1; Fin Tantque

 Tantque T[j] > pivot faire j $\leftarrow$ j-1; Fin Tantque

 Echanger (T, i, j);

Fin Tantque

Retourne i;

Fin

Tri par fusion

Principe:

Inventé en 1948 par Goldstine et Von Neumann en 1948.

- Applique le principe « **diviser pour régner** ».
- Basé sur l'idée que s'il y a deux suites d'éléments triés, il est très facile d'obtenir une troisième suite d'éléments triés, par « interclassement » (ou fusion) des deux précédentes suites. Le principe de ce tri consiste:
 - Diviser l'ensemble ou le tableau en deux sous-ensembles
 - Trier les deux sous ensembles.
 - Fusionner les deux sous ensembles

Tri par fusion

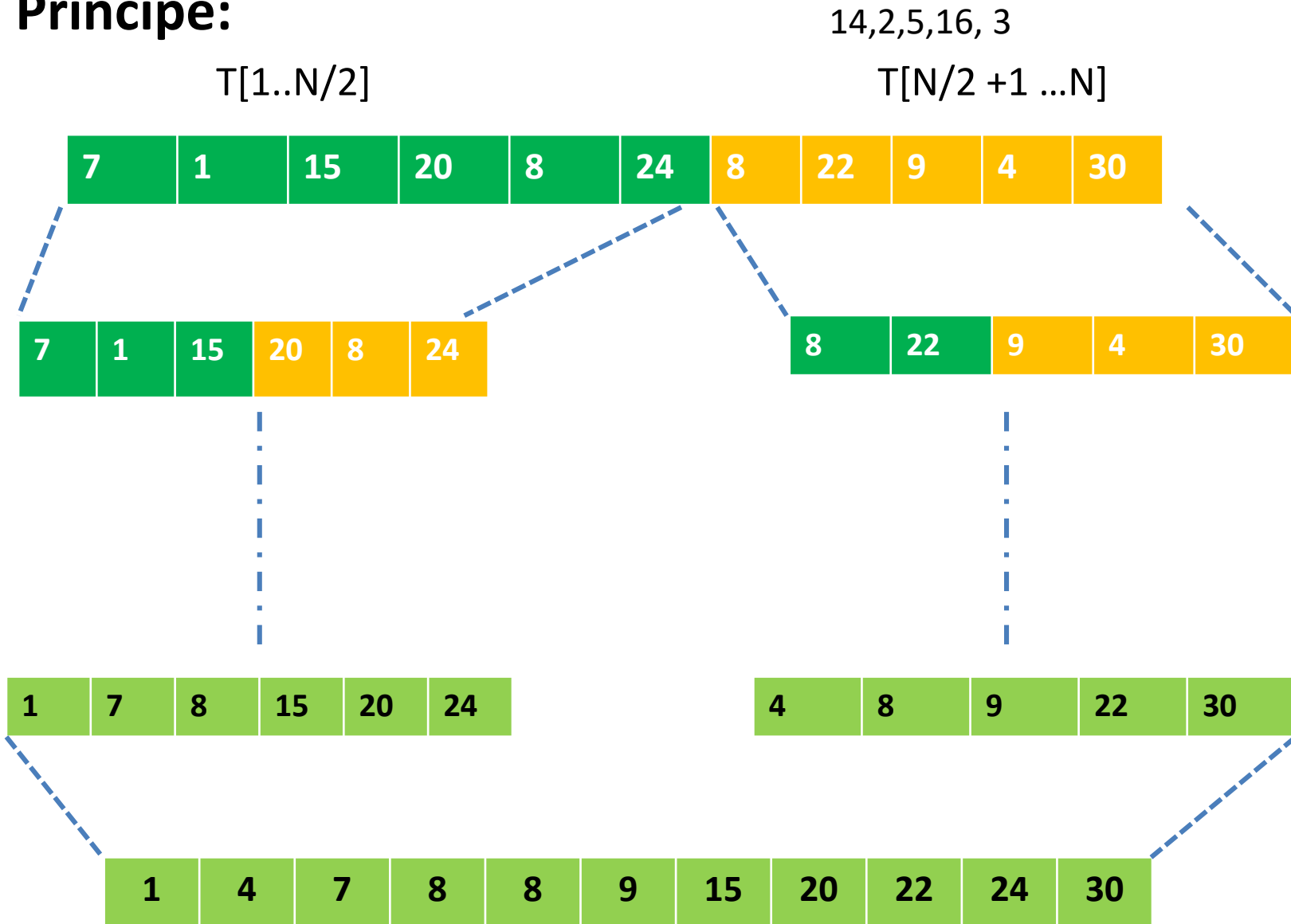
Principe:

Etant donné un tableau (ou une liste) de $T[1, \dots, n]$:

- Si $n = 1$, retourner le tableau T !
- Sinon :
 - Trier le sous-tableau $T[1 \dots n/2]$
 - Trier le sous-tableau $T[n/2 + 1 \dots n]$
 - Fusionner ces deux sous-tableaux. . .
- Il s'agit d'un algorithme “diviser-pour-régner”.

Tri par fusion

Principe:



Tri par fusion(Algorithme)

Fonction Tri_Fusion (Var T : Tableau d'entiers, deb, fin: entier)

mil: entier

Début

Si deb < fin alors

mil $\leftarrow$ (deb + fin)/2

Tri_fusion (T,deb,mil);

Tri_fusion (T, mil+1, fin);

Fusion (T, deb, mil, fin);

Finsi

Fin

Tri par fusion

Procédure Fusionner (var T: Tableau, deb, mil, fin : entier)

T1: Tableau

Début

$i \leftarrow 1$; $j \leftarrow \text{Mil} + 1$;

Pour k de deb à fin faire

Si ($j > \text{fin}$) ou ($i \leq \text{mil}$ et $T[i] \leq T[j]$) alors

$T1[k] \leftarrow T[i]$;

$i \leftarrow i + 1$

Sinon

$T1[k] \leftarrow T[j]$;

$j \leftarrow j + 1$

Fin si

Fin pour

Copier (T, T1, K);

Fin

Propriété des algorithmes de tris

Tri stable:

- Un tri est dit stable s'il préserve l'ordonnancement initial des éléments que l'ordre considère comme égaux.
- Pour définir cette notion, il est nécessaire que la collection à trier soit ordonnancée d'une certaine manière (par exemple pour les listes ou les tableaux).

Tri en place:

- Un tri est dit *en place* s'il n'utilise qu'un nombre très limité de variables et qu'il modifie directement la structure qu'il est en train de trier.
- Ce caractère peut être très important si on ne dispose pas de beaucoup de mémoire.